Power BI Vibes - Client Guide

Client Guide - v0.1.3

What this is

Power BI Vibes is a workflow for building Power BI tools with ChatGPT and GitHub while keeping restricted operational data out of the development conversation when necessary.

You explain the job and the business meaning. ChatGPT handles most repository and Power BI implementation work. Local Power BI validation is reported separately so a committed file is never mistaken for a Desktop-tested result.

First-time setup

Create a private GitHub repository before starting the ChatGPT build. The recommended setup is:

1. go to `https://github.com/new`;
2. choose the correct personal/organization owner;
3. use a short project name;
4. select **Private**;
5. enable **Add README** so the repository starts with a `main` branch and initial commit;
6. create the repository and copy its URL.

For the full instructions, see `docs/CREATE-PRIVATE-REPO.md` in Power BI Vibes.

You do not need to install Git or Power BI Desktop before ChatGPT begins building. Local setup can wait until the first Desktop handoff.

The workflow

1. Create the private GitHub project repository.
2. Connect GitHub to ChatGPT.
3. Give ChatGPT the Power BI Vibes framework URL and your private project URL.
4. Describe what you want the tool to help you do.
5. ChatGPT confirms Power BI is a reasonable fit before creating the Power-BI-specific scaffold.
6. Provide a permitted source representation: approved sample, scrubbed template, metadata-only schema report, or manual schema.
7. ChatGPT creates synthetic development data, a data contract, and a named source parameter so the real source can be connected locally without rewriting downstream logic.
8. Approve the proposed pages/functions/business rules.
9. ChatGPT authors the PBIP project in GitHub. Structural/Desktop checks are marked passed only when the required tooling actually ran; otherwise they remain pending local QA.
10. Prepare the Windows computer, authenticate local GitHub access, clone the project to a short local path, and open it in Power BI Desktop.
11. Follow ChatGPT's specific QA checks.

12 Connect the real operational source through the source parameter and perform the production-data smoke test locally.

13 Continue requesting changes in ordinary language. The project keeps a small learning log so later agent sessions do not repeat solved implementation problems.

Bootstrap prompt

```
I want to build a Power BI tool.

Use this framework:
https://github.com/dudechilling/Power-Bi-Vibes

My private project repository is:
<PASTE YOUR PRIVATE GITHUB REPOSITORY URL>

Read BOOTSTRAP.md in Power-Bi-Vibes and follow it. Do not ask me to share operational data
that I am not permitted to share.
```

Restricted data

If the real source contains information you cannot share, do not upload it just to make development easier. Use a scrubbed/template source or run the local metadata inspector and review its JSON before sharing it.

Schema, screenshots and logs can also reveal restricted information. Treat them according to the same organizational rules as the underlying data.

Synthetic development data

ChatGPT should build a fictional dataset that matches the permitted structure and deliberately covers important edge cases. The synthetic fixture is for development and QA. It does not prove that the real dataset has the same distribution, cleanliness or performance characteristics.

Prepare the Windows computer

At the first Desktop handoff, the normal required local stack is:

- Power BI Desktop;
- Git for Windows;
- PowerShell;
- browser-based GitHub authentication for the private project repository.

GitHub CLI, GitHub Desktop, Python and Visual Studio Code are not required for the normal workflow. SQLite CLI is needed only when locally inspecting a SQLite database schema.

ChatGPT's GitHub connection is separate from local Git authentication. The first local HTTPS operation may open a browser through Git Credential Manager. Sign in with the GitHub account that can access the private repository and complete organization/SSO or two-factor prompts where required.

Configure local Git identity once:

```
git config --global user.name "Your Name"
git config --global user.email "YOUR-GITHUB-EMAIL"
```

Every new client project includes a readiness checker. After cloning, run:

```
.\scripts\check-local-setup.ps1 -RepositoryUrl https://github.com/OWNER/PROJECT.git
```

The checker reports missing dependencies and authentication/access problems without installing software or changing Git configuration. See `docs/WINDOWS-SETUP.md` in Power BI Vibes for the full setup and

troubleshooting guide.

Local Power BI testing

For a first clone:

```
New-Item -ItemType Directory -Path C:\PBI -Force | Out-Null
git clone https://github.com/OWNER/PROJECT.git C:\PBI\PROJECT
cd C:\PBI\PROJECT
```

Before pulling newer changes into a copy that has been opened or saved in Power BI Desktop:

```
git status --short --branch
```

If the tree is clean:

```
git pull --ff-only
```

If files are modified, stop and give the status output to ChatGPT. Desktop saves can modify tracked PBIP/PBIR/TMDL source. Power BI Vibes preserves those edits before resolving local/remote divergence instead of immediately resetting them.

Before the first local open, ChatGPT should check current Microsoft requirements for the chosen PBIP/PBIR format and tell you whether your Power BI Desktop version or preview-feature settings need attention.

Production source

The project should keep its source location/connection behind a named Power Query parameter. After synthetic QA passes, switch that parameter to the approved real source locally.

First refresh can require credentials or a data-source privacy choice. Test refresh, totals, relationships and performance locally. Keep synthetic and production sources separable rather than unnecessarily combining them in one Power Query chain.

Do not send production screenshots, copied records or logs back to ChatGPT when they expose restricted information. Give a sanitized error description or safe screenshot instead.

Project learning

Each private project keeps a small machine-readable learning log for durable observations, confirmed lessons and project patterns. It should capture knowledge a future agent would otherwise rediscover, not every failed command or debugging step.

Private lessons are never copied automatically into the public Power BI Vibes framework. A lesson can become an upstream candidate only after client-specific details are removed and a human reviews the privacy and generality of the proposed rule.

How changes are managed

The private project repository keeps the current usable product on `main` during normal iterative work. ChatGPT makes descriptive checkpoint commits after validated changes. A temporary branch is appropriate for a major experiment on an accepted working product.

Power BI Desktop saves are treated as real source changes. The framework includes a rescue-branch workflow for preserving local edits before synchronizing with newer agent commits.

Repository and document integrity

The framework includes automated integrity checks for required files, version consistency, relative links, YAML structure, client-scaffold mappings, local-setup packaging, and the generated client PDF. The PDF is rebuilt from this Markdown source, checked for a valid page count and substantive content on every page, and rendered before the verified build is published.

These checks reduce silent truncation and packaging drift. They do not replace local Power BI validation of a client project.

What "done" means

A feature is ready when the applicable structural check actually passed, the requested data-bound visuals/functions exist, the rendered layout was actually checked when required, the requested interactions work, and the acceptance steps are current.

If a validator or Desktop check cannot run in the current environment, that check stays pending until it is completed locally.

The production-data smoke test is a separate final check against the approved real source.